

Đặc tả bài toán:

Một phần của hệ thống Northwind quản lý đơn hàng và sản phẩm được mô tả như sau:

Mỗi nhà cung cấp cung cấp một hoặc nhiều sản phẩm, mỗi sản phẩm chỉ thuộc về một nhà cung cấp. Mỗi đơn hàng có thể bán nhiều sản phẩm, mỗi sản phẩm trong đơn hàng có số lượng và giá bán cụ thể. Mỗi sản phẩm có thể thuộc về nhiều đơn hàng khác nhau.

Thông tin chi tiết về các thực thể như sau:

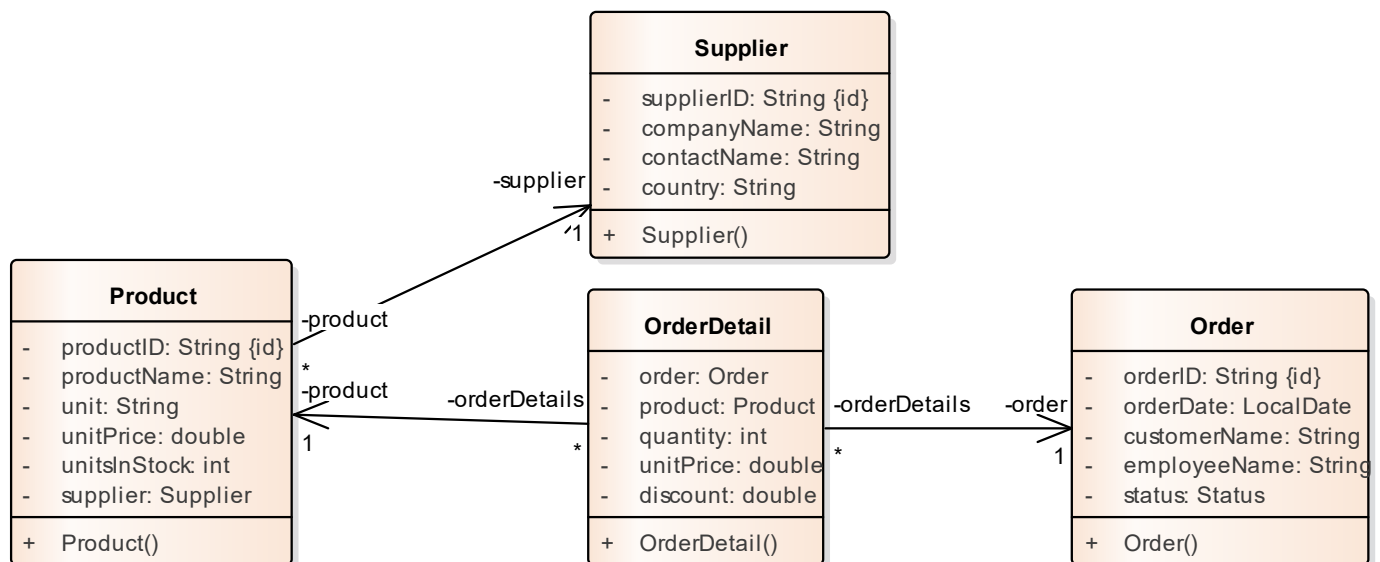
Nhà cung cấp (*Supplier*): Mã số nhà cung cấp (*SupplierID*), tên công ty (*CompanyName*), tên người liên hệ (*ContactName*), quốc gia (*Country*).

Sản phẩm (*Product*): Mã số sản phẩm (*ProductID*), tên sản phẩm (*ProductName*), đơn vị tính (*Unit*), đơn giá (*UnitPrice*) và số lượng tồn kho (*UnitsInStock*).

Đơn hàng (*Order*): Mã số đơn hàng (*OrderID*), tên khách hàng (*CustomerName*), tên nhân viên lập đơn hàng (*EmployeeName*), ngày đơn hàng (*OrderDate*) và trạng thái đơn hàng (*Status* - *COMPLETED*, *PENDING*, *CANCELLED*).

Chi tiết đơn hàng (*OrderDetail*): Ứng với mỗi sản phẩm trong đơn hàng, lưu thông tin về số lượng (*Quantity*), giá bán (*UnitPrice*) và giảm giá (*Discount*).

Class diagram (mô hình lớp)



Graph model (mô hình đồ thị)



Tạo các project tên gồm: STTXX_MSSV_HoTen_May. Dùng ngôn ngữ lập trình JAVA kết nối Neo4j Java Driver và hiện thực các yêu cầu sau:

Tạo database có tên là [tên và mã số của sinh viên] làm bài. Ví dụ: sinh viên tên Lan và có mã số sinh viên là 19898981; thì tên database là Lan19898981.

Câu 1: Thực thi các câu lệnh cypher cho trước để:

- Tạo các unique index trên các thuộc tính id, thuộc tính duy nhất cho tất cả các node.
- Load dữ liệu từ các CSV file cho trước vào database.
- Thiết lập các relationships giữa các node như mô hình đồ thị.

Câu 2: Viết các lớp thực thể theo mô hình lớp trên.

Câu 3: Viết các lớp với các phương thức CRUD theo yêu cầu sau:

a) Tạo Range Index trên thuộc tính tên nhà cung cấp (*CompanyName*) của nhà cung cấp.

Liệt kê danh sách các sản phẩm do một nhà cung cấp cụ thể cung cấp khi biết tên nhà cung cấp, kết quả được sắp xếp theo tên sản phẩm và hỗ trợ phân trang.

+ listProductsBySupplier (companyName: String, int page, int size): List<Product>

Với các điều kiện sau:

- *companyName*: Không được null hoặc rỗng
- *page*: Phải là số nguyên dương ($page \geq 1$)
- *size*: Phải là số nguyên dương ($size \geq 1$)
- Nếu không hợp lệ $\rightarrow$ thông báo lỗi hoặc ném exception

b) Cập nhật thông tin nhà cung cấp khi biết mã số nhà cung cấp (*SupplierID*). Chỉ được phép cập nhật nếu nhà cung cấp đó tồn tại.

+ updateSupplier (supplier: Supplier): boolean

Với các điều kiện sau:

- *supplier*: Không được null
- *SupplierID*: Không được null hoặc rỗng
- Các thuộc tính cập nhật: *CompanyName*: Không được null hoặc rỗng

c) Tính tổng tiền của đơn hàng khi biết mã số đơn hàng (*OrderID*).

+ calculateTotalOrder (orderID: String): double

Tổng tiền của một đơn hàng được tính theo công thức:

$$\text{Tổng tiền} = \sum (\text{Quantity} \times \text{UnitPrice} \times (1 - \text{Discount}))$$

Ràng buộc nghiệp vụ:

- Kiểm tra sự tồn tại của đơn hàng: Nếu không tồn tại $\rightarrow$ trả về 0
- Nếu tồn tại: Tính tổng tiền dựa theo công thức trên.

Câu 4: Viết lớp với phương thức main cho phần kiểm nghiệm. Kiểm thử phủ các yêu cầu trên, xuất kết quả ra màn hình..

----- Hết -----

Lưu ý: Tài liệu sinh viên được phép dùng trong phòng thi:

Trang <https://neo4j.com/docs/>